Smart Parking Access Control (SPAC)

An End-to-End License Plate Recognition System for Automated Vehicle Authorization

Arad Fadaei, Sia Tedy

Table of Contents

Abstract

This report presents the design, implementation, and evaluation of **Smart Parking Access Control (SPAC)**, an end-to-end computer vision system that automates vehicle authorization at parking facilities. The system integrates YOLOv8 object detection for license plate localization, a dual-backend OCR engine with ten preprocessing variants for robust text extraction, and a fuzzy-matching module for resident database lookup. Evaluated on 44 test images from the Kaggle Car Plate Detection dataset, the system achieves a 97.7% plate detection rate, 90.9% decision accuracy, and 85.7% recall for authorized residents. The pipeline is configurable through a centralized YAML specification and exposed through a unified command-line interface that supports all workflow stages from data preparation through evaluation.

Introduction

Problem Statement

Parking access control remains a labor-intensive process at many residential and commercial facilities. Traditional approaches rely on physical access cards, keypads, or manned security checkpoints, all of which incur ongoing operational costs, introduce human error, and inconvenience authorized residents. An automated vision-based system that can identify vehicles by their license plates and grant or deny access in real time therefore offers a practical alternative.

Motivation

Automatic License Plate Recognition (ALPR) has matured significantly in recent years, driven by advances in deep learning-based object detection and optical character recognition. However, assembling a reliable, end-to-end ALPR pipeline for access control requires careful integration of multiple components, including detection, OCR, text normalization, and database matching, each with its own failure modes. This project explores how these components can be composed into a practical, configurable system with measurable performance.

Objectives

The primary objectives of this project are as follows:

1. **Detection:** Train a lightweight object detector to reliably localize license plates in images.
2. **Recognition:** Extract plate text using multi-backend OCR with robust preprocessing.
3. **Matching:** Match extracted text against an authorized resident database using exact and fuzzy matching.
4. **Decision:** Produce access grant/deny decisions with confidence scores.
5. **Evaluation:** Quantify system performance against ground-truth labels.
6. **Tooling:** Provide a reproducible, configurable pipeline accessible via a single CLI.

Previous Work

Automatic license plate recognition is commonly structured as a two-stage pipeline in which a detector first localizes the plate region and a recognition engine then extracts the alphanumeric text. This project follows that general pattern rather than proposing a new detector or OCR architecture from scratch. In particular, the implementation builds directly on Ultralytics YOLO for object detection [1], EasyOCR for scene-text recognition [2], and PaddleOCR as a complementary OCR backend [3], while using the Kaggle Car License Plate Detection dataset as the supervised benchmark for plate localization [4].

The main departure from these prior building blocks is therefore at the system-integration level. Instead of relying on a single OCR pass, the SPAC pipeline adds plate-specific preprocessing variants, dual-backend candidate collection, normalization, plausibility-aware ranking, and fuzzy resident matching so that imperfect OCR output can still be converted into an access decision. Accordingly, the project should be understood as an applied ALPR system built on established open-source components, with its primary contribution lying in the design, orchestration, and evaluation of a complete access-control workflow rather than in the introduction of a new foundational model.

System Architecture

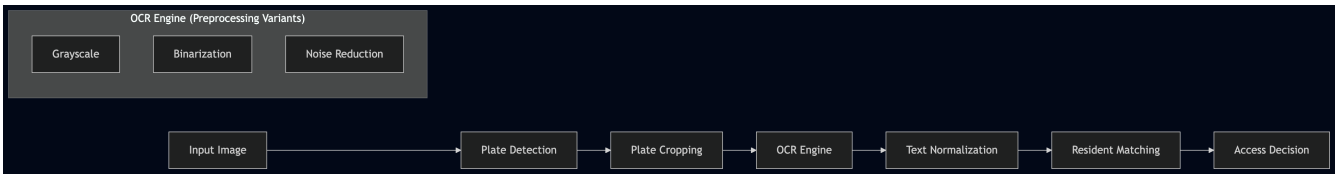

Figure 1. System Architecture Diagram

Method and Contributions

The method used in this project is a modular end-to-end ALPR pipeline tailored to the parking access-control task rather than to plate recognition alone. Starting from an input vehicle image, the system first detects the plate region with YOLOv8n, expands the crop to preserve edge characters, applies multiple OCR-oriented preprocessing variants, extracts candidate text with both EasyOCR and PaddleOCR, and finally converts the recognized plate string into an access decision through exact and fuzzy comparison against a resident database. This design was chosen because the project goal is operational authorization, so the pipeline must remain useful even when OCR is imperfect.

The main contributions of the project are therefore at the system-design level. Relative to the baseline detector-plus-single-OCR pattern described in the previous-work section, the group added configurable crop padding, multiple geometric and contrast preprocessing variants, dual-backend OCR candidate collection, normalization and plausibility-aware ranking of candidate strings, resident-database matching, and end-to-end evaluation artifacts that expose where failures occur. These steps were chosen to make the full workflow more robust and more inspectable than a minimal ALPR prototype, while keeping each stage modular enough to debug and improve independently.

Pipeline Stages

1. **Plate Detection** - A YOLOv8n model localizes the license plate region within the input image and returns a bounding box with an associated confidence score.
2. **Plate Cropping** - The detected region is extracted with configurable padding (14% horizontal, 22% vertical) to preserve plate context while limiting background noise.
3. **OCR Text Extraction** - A dual-backend OCR engine processes the cropped plate through multiple preprocessing variants and ranks candidate strings using OCR confidence and a plausibility heuristic.
4. **Resident Matching** - The extracted text is normalized and compared against a resident database using exact matching followed by fuzzy matching with a 90% similarity threshold.

System Architecture Overview

Each stage produces structured output (bounding box, text, match result), which is aggregated into a per-image result record and serialized to JSON for downstream evaluation.

Dataset

The detector was trained and evaluated using the Kaggle dataset `andrewmvd/car-plate-detection`. This dataset provides vehicle images paired with bounding-box annotations for visible license plates, making it suitable for the plate-localization stage of the SPAC pipeline. Within this project, the dataset serves only the object detection component; OCR and access-control behavior are evaluated later on the crops and structured outputs produced by the full inference pipeline.

Source Data

The raw dataset is stored under `data/raw/car-plate-detection/` and follows a Pascal VOC-style directory structure with separate `images/` and `annotations/` folders. The repository currently contains 433 raw images and 433 XML annotation files, so each image has a corresponding annotation record.

Each annotation file describes the target object through a rectangular bounding box defined by `xmin`, `ymin`, `xmax`, and `ymax` coordinates in image pixel space. For this project, those annotations are treated as a single detection class, `plate`, because the training objective is limited to license plate localization rather than multi-class object recognition.

Sample Images and Ground-Truth Labels

Figure samples from the raw dataset are shown below with their detector ground-truth bounding boxes overlaid. These examples illustrate the variety of scale, background clutter, and viewpoint changes present even within this relatively compact dataset.

Figure 2. Dataset sample with detector ground truth: Cars17

Figure 3. Dataset sample with detector ground truth: Cars50

Figure 4. Dataset sample with detector ground truth: Cars111

The bounding-box labels are provided in Pascal VOC XML files. For example, the `Cars50.xml` annotation encodes a single license plate object with the following coordinates:

```
<annotation>
    <filename>Cars50.png</filename>
    <object>
        <name>licence</name>
        <bndbox>
            <xmin>116</xmin>
            <ymin>55</ymin>
            <xmax>525</xmax>
            <ymax>262</ymax>
        </bndbox>
    </object>
</annotation>
```

This detector ground truth is part of the source dataset itself and is therefore distinct from the manually completed decision-level ground-truth CSV used later during end-to-end evaluation.

Dataset Conversion

Because Ultralytics YOLO expects labels in normalized YOLO format rather than Pascal VOC XML, the raw Kaggle dataset is converted before training. This conversion is handled by `src.data.prepare_kaggle_car_plate_dataset`, which performs the full preprocessing workflow in a reproducible way.

The preparation script parses each XML annotation, extracts the image filename and image dimensions, validates the bounding-box coordinates, clamps them to valid image boundaries, and converts each box into YOLO center-width-height form:

```
(x_center, y_center, width, height)
```

All four values are normalized by the image width and height, and the generated labels are written to text files under the standard YOLO directory structure. The output dataset is stored in `data/processed/car_plate_kaggle/` with paired `images/` and `labels/` directories for each split.

Preparation and Preprocessing for Training

At the project level, detector preparation is intentionally conservative. The main preprocessing steps are annotation validation, coordinate clamping, conversion from Pascal VOC to YOLO format, and deterministic train-validation-test splitting. No custom grayscale conversion, histogram equalization, or handcrafted image enhancement is applied to the detector training images themselves.

Instead, image resizing and tensor normalization are handled by the Ultralytics training pipeline at an input size of `640 x 640`. Likewise, the project does not define a custom augmentation policy in `src.train.train_detector`. This means the experiment does not introduce any project-specific augmentation design; any augmentation applied during training comes from the standard Ultralytics training behavior rather than from manually tuned transforms defined by the group. This keeps the detector experiment simple and reproducible while reserving the heavier image-processing logic for the OCR stage, where it has the greatest impact.

Train-Validation-Test Split

The dataset is shuffled using a fixed random seed of 42 and then divided according to the configured 80/10/10 split ratios. This produces a deterministic partition that can be regenerated later using the same configuration. In absolute terms, the split corresponds to 346 training images, 43 validation images, and 44 test images.

Table 1. Processed Dataset Split

Split	Proportion
Train	80%
Validation	10%
Test	10%

The training split is used for parameter optimization, the validation split is used for checkpoint selection and metric tracking during training, and the test split is reserved for downstream end-to-end inference and evaluation. This separation is important because the detector should be assessed on images that were not seen during optimization.

Dataset Metadata and Reproducibility

After conversion, the preparation stage writes a `dataset.yaml` file in `data/processed/car_plate_kaggle/` that defines the dataset root, the train-validation-test image directories, and the single class name `plate`. This file is the canonical dataset descriptor consumed during YOLO training.

The main dataset preparation parameters are centralized in `configs/default.yaml`, including the raw input path, processed output path, split ratios, random seed, and class name. Keeping these values in configuration rather than hardcoding them in the training script makes the data pipeline reproducible and keeps the report consistent with the implementation.

Training

The training stage focuses exclusively on the license plate detector. Its objective is to learn a single-class object detection model that can localize license plates reliably enough to support the downstream OCR and access-control stages. Because the detector is the entry point to the full pipeline, its role is not only to maximize localization accuracy in isolation, but also to produce crops of sufficient quality for later text recognition.

Model Selection

The detector was trained using the YOLOv8n architecture. This variant was selected as a practical balance between model capacity and computational cost. For a single-class detection problem with a relatively modest dataset size, the nano model is large enough to learn plate localization while remaining lightweight enough for repeated experimentation and practical deployment.

Another advantage of this choice is that the detector can be integrated into the rest of the pipeline without adding significant latency. Since the SPAC system is intended as an end-to-end access-control workflow rather than a standalone detection benchmark, the detector was chosen with whole-system usability in mind rather than benchmark-oriented optimization alone.

Training Configuration

Training was executed through the Ultralytics YOLO training interface using the centralized project configuration. The processed YOLO-format dataset description was provided through `data/processed/car_plate_kaggle/dataset.yaml`, and the detector was initialized from pretrained `yolov8n.pt` weights.

Table 2. Detector Training Configuration

Parameter	Value
Model	`yolov8n.pt`
Dataset YAML	`data/processed/car_plate_kaggle/dataset.yaml`
Epochs	`80`
Image Size	`640`
Batch Size	`16`
Device	`0` (GPU)
Output Name	`detector_train`

The training wrapper itself is intentionally minimal. Its purpose is to expose the core experimental parameters while delegating the optimization loop, validation procedure, checkpointing, and metric reporting to the underlying YOLO implementation. This keeps the project code concise and makes the detector stage easy to reproduce from configuration alone.

Training Procedure

During training, the detector was fine-tuned on a single class, `plate`, using the prepared dataset split and the standard Ultralytics validation cycle at the end of each epoch. Metrics were recorded continuously in `runs/detect/outputs/detector_train/results.csv`, which served as the primary artifact for tracking convergence and selecting the best checkpoint.

Model selection was based on validation `mAP50-95`, since that metric is stricter than `mAP50` and better reflects localization quality across multiple IoU thresholds. After training, the best detector weights can be copied from the training output directory into `models/plate_detector.pt`, which is the path consumed by the inference pipeline.

Training Results

The training log contains 80 recorded epochs. The best validation checkpoint occurred at epoch 57, where the model reached a precision of 0.97216, recall of 0.83673, `mAP50` of 0.87535, and `mAP50-95` of 0.51473. The final epoch achieved slightly lower `mAP50-95`, indicating that the strongest detector snapshot was obtained before the end of the run rather than at the last saved epoch.

Table 3. Detector Snapshot Comparison

Snapshot	Epoch	Precision	Recall	`mAP50` / `mAP50-95`
Best validation checkpoint	`57`	`0.97216`	`0.83673`	`0.87535 / 0.51473`
Final recorded epoch	`80`	`0.97545`	`0.81096`	`0.87023 / 0.46343`

The overall training trajectory shows rapid improvement during the early epochs, followed by a broad performance plateau during the middle and later stages of training. This pattern suggests that the detector converged relatively early and that continued training mainly refined or fluctuated performance rather than producing large late-stage gains. Accordingly, the best checkpoint from epoch 57 is the more appropriate model for deployment within the SPAC pipeline.

Training Artifacts

The detector training stage produces a standard Ultralytics experiment directory containing weights, per-epoch metrics, and supporting visualizations. For this project, the primary artifact directory is `runs/detect/outputs/detector_train/`. The most important outputs are the recorded training metrics in `results.csv` and the best-performing detector checkpoint, which can then be promoted to `models/plate_detector.pt` for inference.

OCR Methodology

The OCR stage is responsible for converting a detected plate crop into a normalized alphanumeric string that can be matched against the resident database. This stage is intentionally more redundant than the rest of the pipeline because OCR quality is highly sensitive to blur, plate skew, contrast variation, specular reflections, and small character size. Rather than committing to a single preprocessing routine or a single recognizer, the system generates multiple plate representations and evaluates them with two OCR backends before selecting the most plausible result.

OCR Design Rationale

The detector returns only a localized crop, not a fronto-parallel or uniformly illuminated plate image. In practice, many failures arise from geometric distortion and local contrast loss rather than complete detector failure. For that reason, the OCR stage was designed around three principles: improve legibility through preprocessing, diversify recognition using multiple engines, and delay final selection until all candidates have been scored.

This design also keeps the OCR component modular. Each preprocessing operation is implemented as a self-contained step, and the sequential pipeline order can be modified without changing the rest of the inference code. Likewise, the backend list is configurable through `configs/default.yaml`, allowing the OCR stage to be simplified or expanded without altering the surrounding pipeline.

Preprocessing Strategy

The OCR engine begins from the padded plate crop produced by the inference pipeline. That crop already preserves some surrounding context by extending the detector bounding box by 14% horizontally and 22% vertically, which reduces the risk of clipping border characters. The OCR module then generates several derived images intended to address common visual failure modes.

Table 4. OCR Preprocessing Operations

Operation	Purpose
Upscaling	Magnifies small plate crops by 2x or 3x using bicubic interpolation so thin strokes become easier to resolve.
Rectification	Estimates the dominant plate orientation from the largest contour and rotates the crop to reduce in-plane skew.
Perspective Flattening	Approximates the plate outline as a quadrilateral and applies a perspective warp to reduce projective distortion.
Grayscale Conversion	Removes color information when it is not useful for recognition and simplifies later enhancement steps.
Bilateral Denoising	Reduces local noise while preserving edges that define character boundaries.
CLAHE	Improves local contrast under uneven illumination without amplifying the entire image uniformly.

Operation	Purpose
Otsu Binarization	Produces a global black-and-white separation when character strokes are already reasonably distinct.
Adaptive Thresholding	Uses local Gaussian thresholding to handle plates with non-uniform lighting or shadows.
Sharpening	Applies a simple enhancement kernel to strengthen stroke edges before recognition.

In addition to these individual variants, the engine also evaluates a sequential preprocessing pipeline whose default order is:

```
upscale -> rectify -> flatten -> grayscale -> denoise -> clahe -> sharpen -> binarize
```

This arrangement reflects a practical assumption: geometric corrections should occur before contrast enhancement and thresholding, while binary conversion should occur late in the process after noise and contrast have already been addressed. The engine does not assume that this sequence is always optimal, which is why the individual variants are preserved and evaluated alongside the full sequential version.

OCR Backends and Text Extraction

Two recognizers are used: EasyOCR and PaddleOCR. EasyOCR is configured with an uppercase alphanumeric allowlist and beam-search decoding, making it well suited to constrained plate text. PaddleOCR serves as a complementary recognizer with a different detection and recognition stack, which helps recover cases where one backend fragments the text or misreads individual characters.

For each image variant, the engine queries both backends and collects multiple candidate strings. When a backend produces several text boxes, the system keeps both the single highest-confidence box and a left-to-right concatenation of all detected boxes. This is important for license plates because characters are sometimes split across multiple OCR regions even though they form one logical identifier.

After recognition, all candidate strings are normalized before ranking. Normalization converts characters to uppercase, strips whitespace, and removes non-alphanumeric symbols. This makes downstream comparison more stable and prevents punctuation or spacing artifacts from being treated as meaningful differences.

Candidate Ranking and Confidence Handling

The final OCR output is selected from the full pool of backend and preprocessing candidates rather than from a single recognizer pass. Each candidate is ranked by a combination of OCR confidence and a lightweight plate-plausibility heuristic. The heuristic rewards strings whose lengths resemble typical plate formats, gives a small bonus to mixed letter-digit strings, and penalizes extremely short outputs or repetitive low-diversity strings that are more likely to be OCR noise.

Formally, candidates are ordered by a tuple of the form `(confidence + plausibility, confidence, text_length)`. This ranking strategy is strong enough to suppress obviously poor reads while remaining simple to explain and modify. Because it is applied only at the selection stage, it can be removed or replaced later without changing the rest of the recognition pipeline.

The OCR confidence threshold in the configuration should be interpreted as a control parameter rather than as a strict rejection gate. In the current implementation, low-confidence text can still be propagated for inspection and downstream evaluation, which is useful during error analysis even when the read would not be trusted in a production system. This behavior reflects the exploratory nature of the project and keeps failure cases visible instead of silently discarding them.

Example Preprocessing Outputs

To make the OCR stage interpretable, the project includes an export utility that saves intermediate images for a selected sample. The example below uses `Cars50`, which produces the final OCR output `PUI8BES` with confidence `0.976521`.

Cars50 Source Images

Figure 5. Original input image

Figure 6. Detected license plate bounding box

Figure 7. Padded plate crop used for OCR

Cars50 Geometric Variants

Figure 8. Upscaled crop

Figure 9. Rectified crop

Figure 10. Perspective-flattened crop

Cars50 Enhancement Variants

Figure 11. Denoised crop

Figure 12. CLAHE-enhanced crop

Figure 13. Otsu-thresholded crop

Figure 14. Adaptive-thresholded crop

Figure 15. Sharpened crop

Figure 16. Rectified grayscale crop

Cars50 Sequential Pipeline Output

Figure 17. Sequential preprocessing pipeline result

Cars50 OCR Export Summary

```
image: data/processed/car_plate_kaggle/images/test/Cars50.png
bbox_xyxy: (125, 70, 526, 255)
detector_conf: 0.877898
final_ocr_text: PUI8BES
final_ocr_conf: 0.976521
```

Together, these artifacts provide a visual explanation of how the OCR stage transforms a detector crop into a final text prediction. They are also useful for debugging because they expose which preprocessing variants preserve character structure and which ones over-process the image.

Evaluation

The evaluation stage measures the SPAC pipeline at two levels: first as an end-to-end inference system that produces detections, OCR outputs, and access decisions for each image; and second as a labeled decision system whose outputs can be compared against manually curated ground truth. This two-level structure is important because a pipeline may still produce detections and OCR text even when the final access decision is wrong, and conversely may deny access correctly even when the recognized plate text is imperfect.

Evaluation Procedure

Pipeline outputs are first serialized to `outputs/predictions/inference_results.json` during inference. The evaluation module then converts those results into a normalized tabular form containing detector status, detector confidence, OCR text, OCR confidence, predicted decision, and matching metadata for each image.

At the base level, evaluation computes aggregate statistics directly from inference output without requiring human labels. These metrics summarize how often the detector produced a bounding box, how often OCR returned non-empty text, how detector and OCR confidences behaved on average, and how many samples were assigned to the access-granted versus access-denied classes.

For labeled evaluation, predictions are merged with the manually completed ground-truth CSV in `outputs/metrics/ground_truth_template.csv` using image filename as the join key. Decision quality is then measured by treating `Access Granted` as the positive class and computing accuracy, precision, recall, F1-score, and the confusion matrix. Plate recognition quality is evaluated separately through exact string matching between the normalized OCR output and the expected plate string. Rows whose expected plate is blank or marked as `UNKNOWN` are excluded from the exact-match calculation so that the OCR metric reflects only labeled plate values.

Base Pipeline Metrics

The current evaluation run contains 44 test images. Of these, the detector successfully localized a plate in 43 images, and OCR returned non-empty text in those same 43 cases. This yields a detection rate and OCR non-empty rate of 97.73% over the test split.

Table 5. Base Pipeline Performance

Metric	Value
Test samples	44
Successful detections	43
Detection rate	97.73%
Non-empty OCR outputs	43
OCR non-empty rate	97.73%
Predicted access granted	9
Predicted access denied	35

Metric	Value
Mean detector confidence	`0.761242`
Mean OCR confidence	`0.587639`

These base metrics indicate that the detection stage is generally reliable on the held-out split and that OCR usually produces some text once a crop is available. However, non-empty OCR output should not be interpreted as correct OCR output, which is why labeled decision and plate metrics are needed as a stricter measure of end-to-end quality.

Decision-Level Results

Using the labeled ground-truth CSV, the pipeline was evaluated on all 44 test images with no unmatched rows. The system achieved an overall decision accuracy of 90.91%, with precision of 66.67%, recall of 85.71%, and F1-score of 0.75 for the positive class `Access Granted`.

Table 6. Decision Classification Metrics

Metric	Value
Labeled samples	`44`
Accuracy	`90.91%`
Precision	`66.67%`
Recall	`85.71%`
F1-score	`0.7500`

Table 7. Confusion Matrix

Term	Count
True positives	`6`
True negatives	`34`
False positives	`3`
False negatives	`1`

This confusion profile shows that the system is better at recovering truly authorized vehicles than at avoiding all incorrect grants. The single false negative indicates that one authorized vehicle was denied access, while the three false positives indicate three denied samples that were incorrectly granted. In an access-control setting, both error types matter, but they represent different operational risks and should be interpreted separately rather than through accuracy alone.

Plate Recognition Results

Plate-text evaluation is intentionally stricter than the access decision metric because it requires an exact match between the normalized OCR string and the expected ground-truth plate. Out of 44 labeled test samples, 39 contained a usable expected plate string after excluding 5 rows marked as blank or `UNKNOWN`. Among those 39 evaluable samples, the exact plate match rate was 28.21%.

Table 8. Plate Recognition Metric

Metric	Value
Plate-labeled samples	39
Excluded blank or unknown labels	5
Exact plate match rate	28.21%

This comparatively low exact-match rate highlights the main weakness of the current system: OCR errors remain common even when plate detection succeeds. That result is consistent with the project's broader design, where fuzzy matching and decision logic are necessary because exact OCR recovery is not yet robust enough to support strict string comparison alone.

Ground-Truth Construction

End-to-end evaluation uses a second layer of ground truth beyond the detector annotations. After inference, the project generates a CSV template containing one row per test image, and that file is then completed manually with the expected access decision and expected normalized plate string. This makes it possible to evaluate not only whether a plate was localized, but also whether the final access-control outcome was correct.

A representative excerpt of the manually completed file is shown below.

```
image_path,expected_decision,expected_plate,notes
data/processed/car_plate_kaggle/images/test/Cars111.png,Access Granted,MH20EE7598,
data/processed/car_plate_kaggle/images/test/Cars112.png,Access Granted,SHAKNBK,
data/processed/car_plate_kaggle/images/test/Cars138.png,Access Denied,TN02BL9,
data/processed/car_plate_kaggle/images/test/Cars50.png,Access Denied,PUI8BES,
```

This manual labeling step is necessary because the source Kaggle dataset provides detector boxes but does not provide the resident authorization labels required by the SPAC decision task.

Qualitative Examples

Quantitative metrics summarize overall behavior, but individual outputs are also useful for understanding where the pipeline succeeds and where it fails. The examples below show four representative cases taken from the held-out test split.

Figure 18. Qualitative example: correct grant with successful OCR and matching (Cars17)

In `Cars17`, the detector localizes the plate successfully, OCR recovers `YSX213`, and the matcher returns the correct authorized resident record. This is a representative end-to-end success case.

Figure 19. Qualitative example: correct deny with successful OCR but no authorized match (Cars50)

In `Cars50`, OCR correctly recovers `PUI8BES`, but the plate is not found in the authorized resident database, so the system correctly denies access. This case shows that accurate OCR does not necessarily imply an access grant.

Figure 20. Qualitative example: denied case caused by missed detection (Cars138)

In `Cars138`, the detector fails to produce a bounding box, so no OCR text is generated and the pipeline defaults to denial. This is the clearest example of an upstream detector failure propagating directly to the final decision.

Figure 21. Qualitative example: incorrect grant despite correct OCR (Cars293)

In `Cars293`, OCR recovers the expected plate string `GBI8TCE`, but the system still predicts `Access Granted` while the manual label is `Access Denied`. This illustrates that the access-control task depends not only on recognition quality but also on the correctness of the downstream matching and labeling assumptions.

Comparison with Previous Work

Direct numerical comparison with prior ALPR work is limited in this report because published

systems often evaluate on different datasets, use different plate formats, or stop at text recognition without modeling an access-control decision. Even so, the overall structure of the SPAC pipeline is consistent with the broader ALPR literature summarized in the previous-work section: a detector proposes the plate region, OCR extracts text, and downstream logic resolves the operational decision.

Relative to those established building blocks, the present system performs strongly at plate localization on the current benchmark and achieves useful decision-level accuracy, but it does not yet deliver equally strong exact plate recovery. That comparison is important because it clarifies the project's contribution: the report does not claim a new state-of-the-art ALPR model, but rather demonstrates a complete and inspectable smart parking workflow built on modern open-source components and evaluated end to end.

Evaluation Artifacts

The evaluation stage writes two primary artifacts. The aggregate summary is stored in `outputs/metrics/evaluation_summary.json`, while the merged per-image inspection table is stored in `outputs/metrics/evaluation_rows.csv`. The second file is particularly useful for qualitative analysis because it exposes where failures originate, including missed detections, low-confidence OCR outputs, incorrect exact matches, and decision-level mismatches.

Project Repository and Tooling

Found here: https://github.com/Downmoto/CV_SPAC

The SPAC project is organized as a modular Python repository in which each major pipeline stage is implemented as a separate package. This structure keeps data preparation, model training, inference, OCR, matching, and evaluation concerns isolated from one another while still allowing them to be orchestrated through a single command-line entrypoint. As a result, the repository supports both end-to-end execution and targeted experimentation on individual stages.

Repository Structure

At the top level, the repository separates source code, configuration, data artifacts, trained models, generated outputs, and report materials. The most important directories are shown below.

```
CV_proj/
├── configs/                    # centralized configuration files
│   └── default.yaml             # default paths, thresholds, and hyperparameters
├── data/                       # raw, processed, and database data assets
│   ├── db/                      # resident access database csv files
│   ├── processed/               # YOLO-formatted train, validation, and test data
│   └── raw/                     # original Kaggle images and XML annotations
├── docs/                       # report sources and generated evaluation tables
│   ├── evaluation_tables.md     # generated markdown summary tables
│   └── report/                  # AsciiDoc report files, includes, and report
images
├── models/                     # promoted model checkpoints used for inference
│   └── plate_detector.pt        # trained detector weights used by the pipeline
├── outputs/                    # runtime outputs from inference, evaluation, and
debugging
│   ├── demo/                    # rendered visual predictions and examples
│   ├── metrics/                 # evaluation summaries, csv rows, and ground-truth
files
│   ├── predictions/             # serialized end-to-end inference outputs
│   └── preprocessing_steps/     # exported OCR preprocessing variants for
inspection
├── runs/                       # Ultralytics training artifacts and experiment
logs
│   └── detect/                  # detector training runs and checkpoint
directories
├── src/                        # implementation code for the SPAC pipeline
│   ├── data/                    # dataset preparation and resident db generation
│   ├── eval/                    # evaluation logic and report table generation
│   ├── infer/                   # end-to-end inference pipeline and exports
│   ├── matching/                # exact and fuzzy plate matching logic
│   ├── ocr/                     # OCR engine and preprocessing strategy
│   ├── train/                   # detector training wrapper and training utilities
│   ├── utils/                   # shared helper functions
│   └── main.py                  # top-level workflow runner cli
```

```
├──── README.md                  # project overview and usage instructions
└──── requirements.txt           # Python dependency list
```

This layout is intentionally straightforward rather than framework-heavy. Source files are kept under `src/`, report assets are grouped under `docs/report/`, and generated artifacts are separated into `outputs/`, `runs/`, and `models/` so that experiment products do not become entangled with implementation code.

CLI Workflow Runner

The central entrypoint for the repository is `src.main`, which acts as a workflow runner for all major project stages. Rather than requiring each stage to be invoked manually with different paths and parameters, the CLI reads from `configs/default.yaml` and dispatches the requested stage-specific modules with the appropriate arguments.

This design provides two practical benefits. First, it reduces repetition by keeping all default paths and hyperparameters in a single configuration file. Second, it makes the project easier to reproduce because the same command structure can be used for partial runs such as inference-only experiments or complete end-to-end execution.

Table 9. CLI Actions

Flag	Function	Invoked Component
`--download`	Download and unzip the Kaggle dataset	Kaggle CLI
`--prepare`	Convert raw annotations into YOLO format	`src.data.prepare_kaggle_car_plate_dataset`
`--seed-db`	Generate the resident access database	`src.data.create_sample_db`
`--train`	Train the plate detector	`src.train.train_detector`
`--infer`	Run end-to-end inference	`src.infer.run_inference`
`--eval`	Evaluate predictions and decisions	`src.eval.evaluate_pipeline`
`--report`	Generate report-ready metric tables	`src.eval.generate_report_tables`
`--all`	Execute the full workflow in sequence	Combined pipeline

The same entrypoint also supports targeted overrides, such as evaluating with a custom ground-truth CSV or running inference on a single image instead of the default test directory. This makes the CLI suitable both for reproducible report generation and for controlled debugging of individual samples during development.

Module Organization

The `src/` directory is divided into task-specific subpackages that mirror the major stages of the SPAC pipeline. This organization keeps the codebase easy to navigate and makes it clear which module is responsible for each stage of the workflow.

Table 10. Source Modules

Module	Responsibility
`src.data`	Dataset conversion, Kaggle data preparation, and resident database generation.
`src.train`	Detector training and training-stage wrappers around Ultralytics YOLO.
`src.infer`	End-to-end inference pipeline, image processing exports, and detector-to-decision execution.
`src.ocr`	OCR engine, preprocessing variants, backend coordination, and candidate ranking.
`src.matching`	Exact and fuzzy matching against the resident plate database.
`src.eval`	Metric computation, ground-truth integration, and report-table generation.
`src.utils`	Shared utility helpers such as plate-text normalization.

This separation is especially useful for a report-driven project because each subsystem can be described, evaluated, and revised independently. For example, OCR preprocessing can be improved without changing detector training logic, and evaluation rules can be updated without touching the inference pipeline itself.

Configuration and Artifacts

The repository relies on `configs/default.yaml` as its primary configuration source. That file stores dataset paths, training settings, inference thresholds, OCR backend choices, evaluation settings, and artifact destinations. Using a centralized configuration file avoids scattering hardcoded values across multiple scripts and helps keep experimental settings synchronized across stages.

Generated outputs are also stored in a stage-aware way. Training artifacts are written under `runs/detect/outputs/`, inference outputs under `outputs/predictions/` and `outputs/demo/`, evaluation summaries under `outputs/metrics/`, and OCR inspection images under `outputs/preprocessing_steps/` or the report-local image copies in `docs/report/images/steps/`. This separation makes it easier to trace which files belong to which stage and simplifies report preparation.

Supporting Tooling

In addition to the main workflow runner, the repository includes smaller utilities that support experimentation and reporting. Examples include the preprocessing export script for OCR inspection, the ground-truth template generator for manual labeling, and the report-table generator that converts raw metrics into compact summary tables.

Taken together, these tools make the repository more than a single inference script. They provide a lightweight but complete experimentation environment in which data preparation, detector training, OCR debugging, evaluation, and report production can all be managed from the same codebase.

Code Submission and Reproduction

The project code is submitted through the public repository link shown at the start of this section. To run the project, a Python environment should first be created and the dependencies in `requirements.txt` installed. If the Kaggle dataset needs to be downloaded through the workflow runner, the Kaggle CLI and local Kaggle API credentials must also be configured in the environment.

Setup and Run Instructions

```
python -m pip install -r requirements.txt
python -m pip install kaggle
python -m src.main --help
```

After setup, the main workflow stages can be executed from the repository root with the same CLI used throughout the project. For the full decision-level evaluation workflow, two additional points are important. First, the ground-truth CSV is not supplied by the source dataset and must be generated from inference output and then completed manually. Second, the resident database is created twice in practice: once as an initial sample database to support the first end-to-end inference pass, and once again after manual ground-truth labeling so that the final database is based on the cleaned `expected_plate` values rather than on provisional inputs. In the submitted repository, `data/db/residents.csv` already contains the first-pass sample database used for the initial pipeline run, so the reproduction steps below begin from inference and then show how the final labeled workflow is produced.

Example Commands

```
python -m src.main --prepare
python -m src.main --train
python -m src.main --infer
python -m src.eval.create_ground_truth_template --inference-json
outputs/predictions/inference_results.json --output-csv
outputs/metrics/ground_truth_template.csv
python -m src.data.create_sample_db --ground-truth-csv
outputs/metrics/ground_truth_template.csv --output-csv data/db/residents.csv
python -m src.main --infer --eval --report --use-ground-truth
python -m src.main --all #runs entire process from start to finish (not ground truth
generation or second DB pass)
```

In the evaluation workflow, the ground-truth file created by `src.eval.create_ground_truth_template` starts as a blank template with the columns `image_path`, `expected_decision`, `expected_plate`, and `notes`. That file must then be labeled manually by reviewing each test image and filling in the expected access decision and the normalized expected plate string. Only after this manual step can the final labeled evaluation be run.

The resident database workflow is intentionally two-stage. Before final labeling, an initial sample database can be generated for the first decision pass by using a provisional seed CSV together with `--fallback-to-ocr`, which allows OCR output from inference to populate temporary resident

records. After the ground-truth CSV has been completed manually, the database should be generated again without OCR fallback so that the resident records reflect the labeled `expected_plate` values used for the final evaluation.

The default paths, thresholds, and stage settings are read from `configs/default.yaml`. This keeps the run procedure reproducible and allows the full pipeline or individual stages to be rerun without modifying source code. Important generated outputs include detector training artifacts under `runs/detect/outputs/`, inference predictions under `outputs/predictions/`, evaluation summaries under `outputs/metrics/`, and report-ready tables under `docs/evaluation_tables.md`.

Limitations and Challenges

The current SPAC pipeline demonstrates that a compact end-to-end access-control system can be built and evaluated successfully, while also highlighting several areas where additional refinement would further improve robustness. These challenges are not confined to a single module; instead, they emerge from the interaction between dataset scale, detector behavior, OCR quality, and the matching logic that converts recognized text into a final authorization decision.

Dataset Scope and Generalization

The detector and OCR workflow were developed and evaluated on a relatively small dataset derived from a single car-plate corpus. Although the train, validation, and test split is deterministic and well suited for reproducible experimentation, it does not yet cover the full diversity of plate layouts, camera angles, motion blur, lighting conditions, weather effects, and regional formatting that would appear in a real parking environment. Accordingly, the reported metrics should be interpreted as strong evidence that the pipeline works on the current benchmark, with future gains likely to come from testing on broader and more varied operational data.

Another practical limitation is that the access-control database used in the project is intentionally small and synthetic. This makes it appropriate for demonstrating the matching and authorization logic in a controlled way, while leaving room for future evaluation on larger and noisier resident registries, duplicate-like entries, and more realistic update and maintenance workflows.

OCR and Low Plate Matching Performance

The main area with the clearest opportunity for improvement is plate-text recovery. While the detector successfully localized a plate in 43 of 44 test images and OCR returned a non-empty string in those same 43 cases, the exact plate match rate was only 28.21% across the 39 samples with usable plate labels. This gap shows that producing text is not the same as producing correct text, and it indicates that OCR remains the dominant bottleneck in the current pipeline.

Several factors contribute to this result. License plates occupy a small region of the image, so even a correct detection may still produce a crop with limited character detail. Perspective distortion, motion blur, shadows, reflections, low contrast, and compression artifacts can all degrade OCR quality before recognition begins. In addition, visually similar characters such as `0/O`, `1/I`, `5/S`, and `8/B` are especially difficult to separate when the crop is imperfect. The preprocessing variants and dual-backend OCR strategy already improve robustness, but they do not yet fully eliminate these ambiguities.

This limitation also explains why the project relies on fuzzy matching instead of exact string comparison alone. The authorization stage can still achieve useful decision accuracy because approximate text matches are often sufficient to recover the intended resident record. At the same time, this design introduces a predictable tradeoff: when OCR output is weak or partially incorrect, the matcher may still return a plausible but wrong resident, contributing to incorrect grants. The evaluation results reflect this behavior, with three false positives and one false negative at the decision level, while also showing that the overall decision pipeline remains effective despite imperfect text recovery.

System-Level Tradeoffs

The current implementation prioritizes transparency and modularity over aggressive optimization. Each stage writes interpretable artifacts, exposes intermediate outputs, and can be run independently through the CLI, which is useful for debugging and for academic analysis. The tradeoff is that the system is not yet optimized for low-latency deployment, large-scale throughput, or continuous real-time operation.

More broadly, the pipeline still depends on hand-designed thresholds and heuristics in both OCR candidate ranking and resident matching. Those rules are reasonable for a compact prototype and make the system easier to inspect and explain, but they may require retuning when the plate format, imaging conditions, or resident database characteristics change. Future improvements should therefore focus on stronger OCR specialization, broader data coverage, and more systematic calibration of the matching stage so that the promising end-to-end behavior observed here can extend more reliably beyond the current evaluation setting.

Conclusion

Outcome and Reflection

Overall, the method achieved the expected outcome at the system level but not yet at the exact-recognition level. The pipeline successfully delivers a working access-control workflow with strong plate detection performance and high decision accuracy on the held-out test split, which shows that the chosen architecture is effective for the project goal. At the same time, the low exact plate match rate confirms that OCR remains the main limitation of the current design. This reflection is important because it shows that the project did not fail as an end-to-end prototype; rather, it succeeded in identifying which component now deserves the most improvement, namely OCR robustness under difficult viewing conditions.

This project showed that an end-to-end smart parking access-control pipeline can be built as a coherent and practical computer vision system using a lightweight detector, a multi-variant OCR stage, and a database-driven matching module. Across the report, the system demonstrated strong plate localization performance, reliable generation of non-empty OCR outputs, and a decision pipeline that achieved useful access-control accuracy on the held-out test split. These results indicate that the overall architecture is sound and that the modular design choices made throughout the project were effective for both experimentation and evaluation.

An important outcome of the project is that the strongest remaining challenge is now clearly identified. Detection is already robust on the current benchmark, while OCR accuracy remains the main factor limiting exact plate recovery and, by extension, the quality of downstream matching. This is a constructive result rather than a setback, because it narrows the path for future work and shows that further gains are likely to come from OCR-focused improvements rather than a full redesign of the system.

The SPAC repository also provides a useful engineering foundation for continued development. Its unified CLI, centralized configuration, reproducible dataset preparation, exportable preprocessing steps, and evaluation artifacts make the pipeline easy to inspect, rerun, and extend. As a result, future work can build directly on the existing implementation by expanding the dataset, improving OCR specialization, calibrating the fuzzy matcher more systematically, and testing the system in more realistic parking scenarios.

Overall, the project achieves its primary goal: it delivers a working and well-structured prototype for automated vehicle authorization while producing clear evidence about where the next performance improvements should be pursued. In that sense, the project serves not only as a successful end-to-end implementation, but also as a strong baseline for more capable future smart parking systems.

References

[1] G. Jocher, A. Chaurasia, and J. Qiu, **Ultralytics YOLO**. GitHub repository, Ultralytics, 2023. Available: https://github.com/ultralytics/ultralytics. Accessed: Apr. 6, 2026.

[2] JaidedAI, **EasyOCR**. GitHub repository, 2020. Available: https://github.com/JaidedAI/EasyOCR. Accessed: Apr. 6, 2026.

[3] C. Cui, T. Sun, M. Lin, T. Gao, Y. Zhang, J. Liu, X. Wang, Z. Zhang, C. Zhou, H. Liu, Y. Zhang, W. Lv, K. Huang, Y. Zhang, J. Zhang, J. Zhang, Y. Liu, D. Yu, and Y. Ma, "PaddleOCR 3.0 Technical Report," arXiv:2507.05595, 2025.

[4] Andrew MVD, **Car License Plate Detection**. Kaggle dataset, updated approximately 2020. Available: https://www.kaggle.com/datasets/andrewmvd/car-plate-detection. Accessed: Apr. 6, 2026.

Declaration and Group Contributions

We, Arad Fadaei and Sia Tedy, declare that the attached project is entirely our own work and has been completed in accordance with the Seneca Academic Policy. We have not copied or reproduced any part of this assignment, either manually or electronically, from any unauthorized source, including but not limited to AI tools, homework-sharing websites, or other students' work, unless explicitly cited as references. We have not shared our work with others, nor have we received unauthorized assistance in completing this project.

The contribution summary below should reflect the final agreed division of work before submission.

Table 11. Group Contribution Summary

Name	**Task(s)**
Arad Fadaei	Co-developed the initial project baseline with the group and took primary responsibility for repository structure, workflow CLI integration, dataset preparation, detector training, evaluation scripts, and final report integration.
Sia Tedy	Co-developed the initial project baseline with the group and took primary responsibility for OCR experimentation support, manual ground-truth labeling, qualitative result review, database and decision validation, and supporting report and presentation materials.